

Deep Learning Basics

From Neurons to Neural Networks

Mr. Shridhar Galande

Agenda

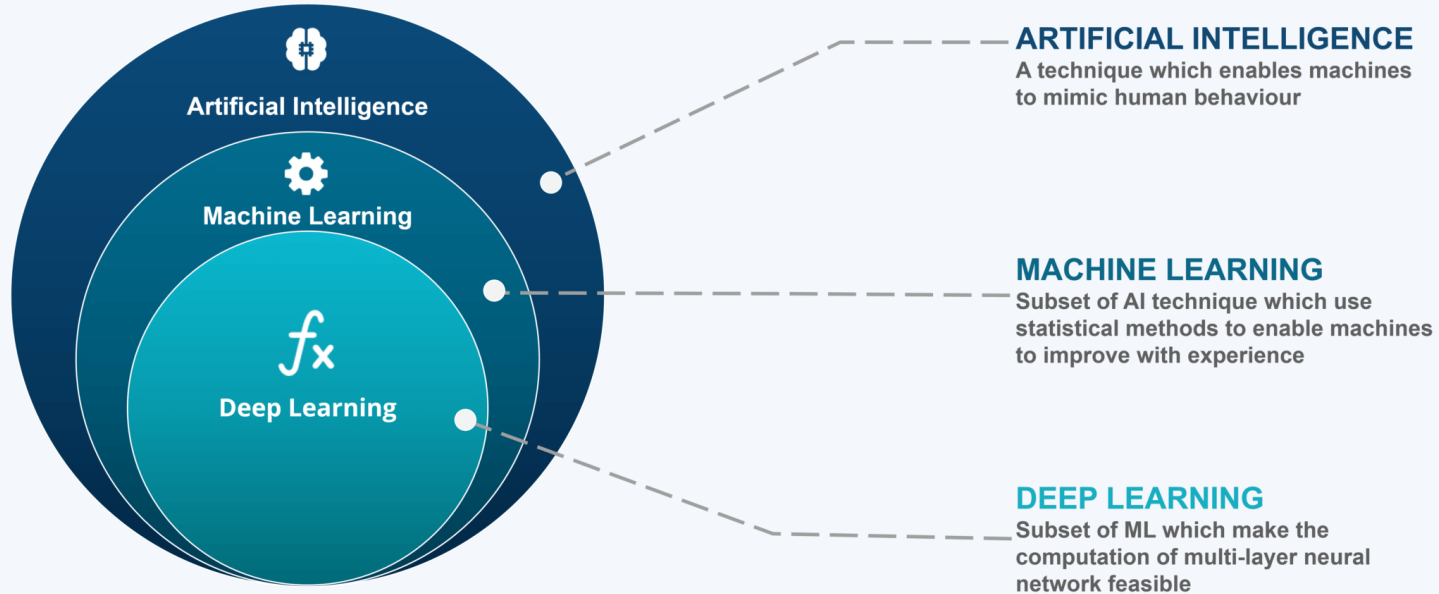
Part 1 Foundations of Neural Networks

Part 2 Types of Neural Networks

Foundations of Neural Networks

- ❖ Introduction to Deep Learning
- ❖ Biological Neuron vs Artificial Neuron
- ❖ Artificial Neural Networks (ANN)
- ❖ Forward Propagation & Backpropagation
- ❖ Activation Functions & Loss Functions
- ❖ Gradient Descent & Optimizers
- ❖ Regularization

What is Deep Learning?



***Deep Learning** is a branch of Machine Learning that uses neural networks with multiple layers to learn patterns, relationships, and features directly from data, enabling computers to solve complex tasks such as image recognition, language understanding, and speech processing.*

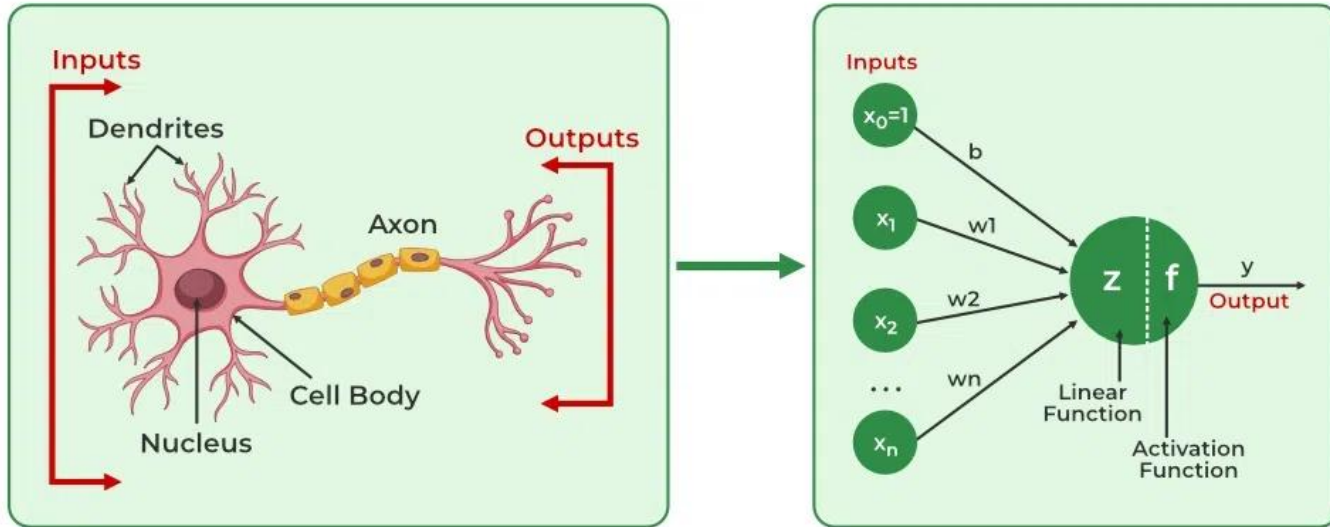
Why Deep Learning?

Deep Learning has transformed Artificial Intelligence by enabling machines to learn complex patterns directly from data without extensive manual feature engineering.

Key Reasons for Its Success

- **Automatic Feature Learning** : Learns relevant features directly from raw data.
- **Handles Unstructured Data** : Excels at processing images, text, audio, and video.
- **High Accuracy** : Delivers state-of-the-art performance on complex tasks.
- **Scalability** : Improves as more data and computing power become available.
- **Real-World Impact** : Powers technologies such as self-driving cars, facial recognition, virtual assistants etc.

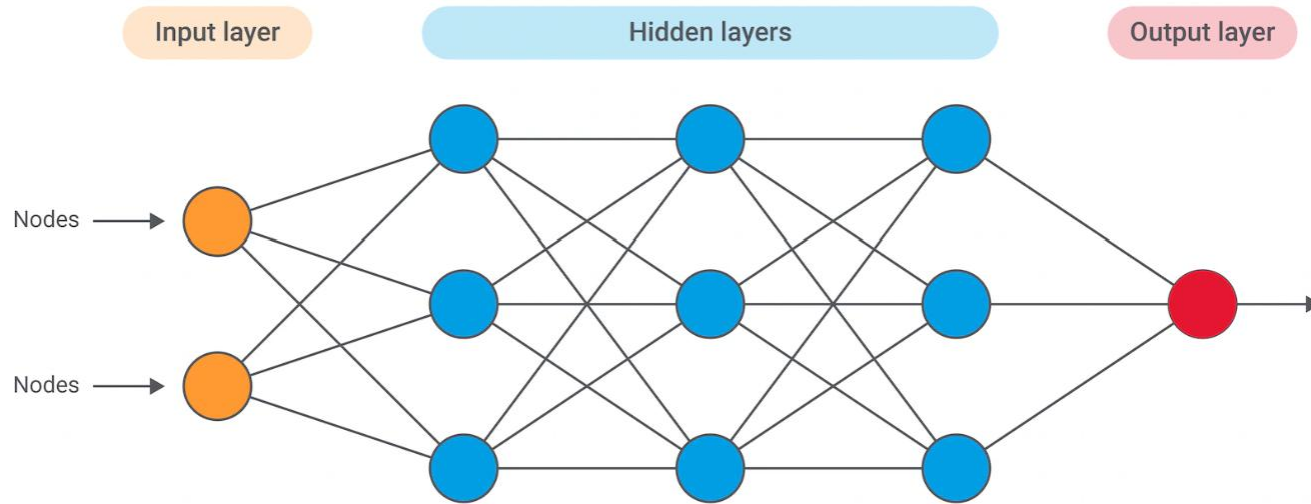
From Biological Neuron to Artificial Neuron



Source: <https://www.geeksforgeeks.org/deep-learning/neural-networks-a-beginners-guide/>

💡 Think of a neuron as a tiny decision-maker that learns which inputs to pay attention to.

Artificial Neural Network



💡 **ANN** is a computational model inspired by the human brain, consisting of interconnected artificial neurons that process and learn from data. These neurons are organized into input, hidden, and output layers, enabling the network to identify complex patterns and make predictions.

Building a Neural Network — Layer by Layer

1

Input Layer

Raw data:
pixels, numbers,
text tokens

2

Hidden Layer 1

Learns low-level
features:
edges, patterns

3

Hidden Layer 2

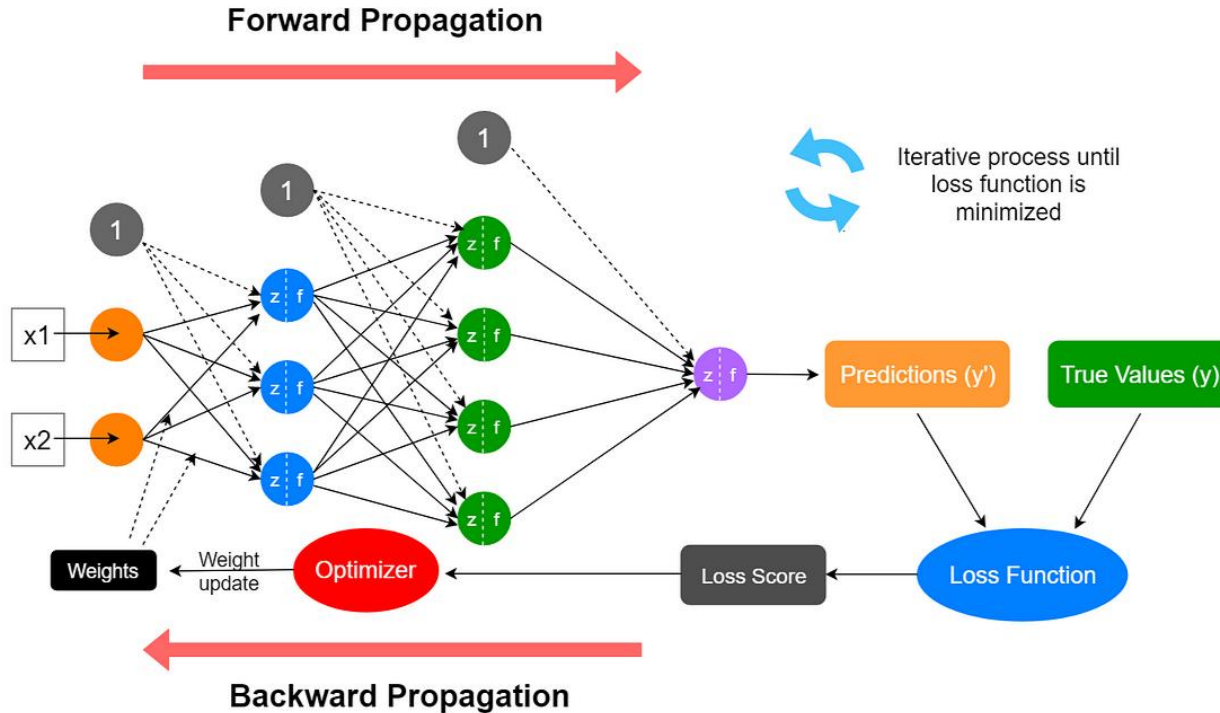
Combines features:
shapes, structures

4

Output Layer

Final prediction:
class label,
probability

Forward and Backward Propagation



Forward and Backward Propagation

Forward Propagation

- Forward propagation is the process of passing input data through the neural network to generate predictions.
- Each neuron computes a weighted sum of inputs and applies an activation function.
- Information flows from the:
input layer → **hidden layer(s)** → **output layer**.
- The final output represents the model's prediction.
- It is used to calculate the error (loss) by comparing predictions with actual values.

Forward Propagation: Produces predictions.

Backward Propagation

- Backpropagation is the process of learning from errors made during prediction.
- The error (loss) is propagated backward from the output layer to the input layer.
- It calculates the contribution of each weight to the prediction error using gradients. Weights and biases are updated using Gradient Descent to reduce the loss.
- This process is repeated over multiple iterations until the model achieves better accuracy.

Backpropagation: Improves predictions by updating weights.

Backpropagation — How the Network Learns



Forward Pass

Input flows through all layers to produce a prediction



Compute Loss

Compare prediction to true label using loss function



Backward Pass

Chain rule calculates gradient of loss for each weight



Update Weights

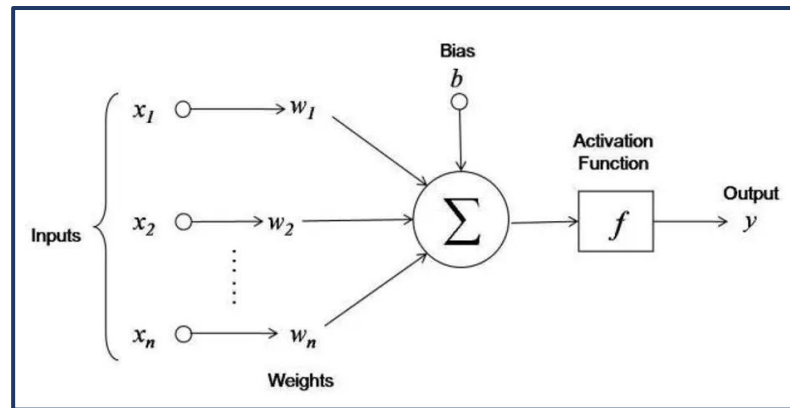
Optimiser adjusts weights in direction that reduces loss

Activation Functions

An **Activation Function** determines whether a neuron should be activated by transforming the weighted sum of inputs into an output signal. It introduces **non-linearity**, enabling neural networks to learn complex patterns.

Why Do We Need Activation Functions?

- Introduce non-linearity into neural networks
- Help learn complex patterns and decision boundaries
- Enable deep networks to solve real-world problems
- Improve model performance and accuracy



Activation functions enable neural networks to learn non-linear and complex patterns, making deep learning powerful for real-world applications.

Common Activation Functions

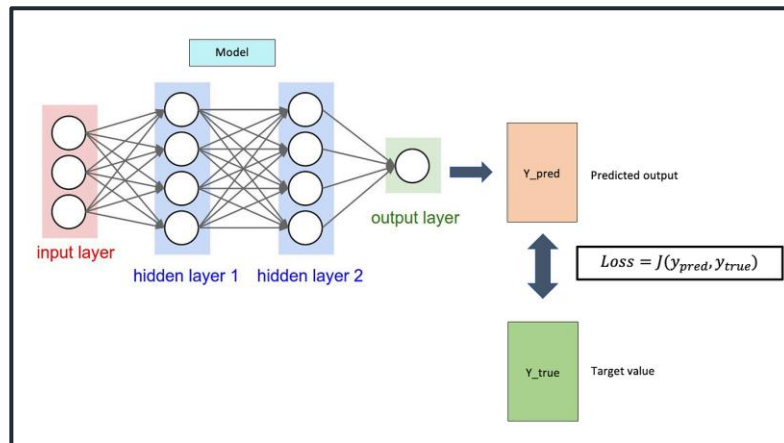
Activation Function	Formula	Output Range	Advantages	Use Cases
Linear	$f(x)=x$	$(-\infty, \infty)$	Simple and interpretable	Regression Output Layer
Sigmoid	$\sigma(x)=1/(1+e^{-x})$	$(0, 1)$	Produces probabilities	Binary Classification
Tanh	$\tanh(x)$	$(-1, 1)$	Zero-centered outputs	Hidden Layers (older networks)
ReLU	$\max(0,x)$	$[0, \infty)$	Fast and efficient	Most Deep Learning Models
Softmax	$e^{x_i}/\sum e^{x_j}$	$(0, 1)$ and sums to 1	Generates class probabilities	Multi-Class Classification

Loss Functions

A **Loss Function** measures the difference between the model's predicted output and the actual (true) output. It helps determine how well the neural network is performing.

Why Do We Need Loss Functions?

- Quantifies prediction errors
- Guides the learning process
- Helps optimize model parameters (weights and biases)
- Lower loss indicates better model performance



Loss Function = Measure of Error **Goal of Training = Minimize Loss**
Lower Loss → Better Predictions → Better Model Performance

Common Loss Functions

Mean Squared Error (MSE)

Used for **Regression Problems**

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Examples: House Price Prediction, Sales Forecasting

Binary Cross-Entropy

Used for **Binary Classification**

$$L = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

Examples: Spam Detection, Fraud Detection

Categorical Cross-Entropy

Used for **Multi-Class Classification**

$$L = - \sum y_i \log(\hat{y}_i)$$

Examples: Image Classification, Language Identification

 **Loss Function = Measure of Error**  **Goal of Training = Minimize Loss**
 **Lower Loss → Better Predictions → Better Model Performance** 

Loss Functions — A Worked Example

Example: predicting whether an email is spam

True label $y = 1$ (it is spam). Model predicts $\hat{y} = 0.8$

Mean Squared Error (regression-style)

- $MSE = (y - \hat{y})^2 = (1 - 0.8)^2 = 0.04$

Binary Cross-Entropy (classification-style)

- $BCE = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})] = -\log(0.8) \approx 0.223$

If the model had instead predicted $\hat{y} = 0.4$ for this same spam email, MSE only rises to 0.36 — but Cross-Entropy jumps to about 0.92, penalizing the confident wrong turn far more sharply.

This is why Cross-Entropy, not MSE, is the standard loss for classification problems.

Optimizers

An **Optimizer** is an algorithm that adjusts the weights and biases of a neural network to minimize the loss function and improve model performance.

Why Do We Need Optimizers?

- Minimize prediction errors (loss)
- Speed up model training
- Find the optimal weights and biases
- Improve convergence and accuracy

Weight Update Rule

$$w_{new} = w_{old} - \eta \frac{\partial J}{\partial w}$$

Where:

- **w** = Weight
- **η (eta)** = Learning Rate
- **J** = Loss/Cost Function

- 📌 Loss Function tells us "how wrong" the model is.
- 📌 Backpropagation calculates gradients.
- 📌 Optimizer uses gradients to update weights and reduce loss.

Optimizer = Engine that drives learning in a Neural Network. 🚀

Common Optimizers

Optimizer	Description	Advantages
Batch Gradient Descent	Uses the entire dataset to update weights	Stable but computationally expensive
Stochastic Gradient Descent (SGD)	Updates weights after each training example	Faster learning and less memory usage
Mini-Batch Gradient Descent	Uses small batches of data for updates	Most commonly used approach
Momentum	Adds previous gradients to current update	Faster convergence and reduced oscillations
AdaGrad	Adapts learning rate for each parameter	Good for sparse data
RMSprop	Uses moving average of squared gradients	Handles varying learning rates effectively
Adam	Combines Momentum and RMSprop	Most popular optimizer in Deep Learning
AdamW	Adam with better weight regularization	Widely used in Transformers and LLMs

Gradient Descent Optimizers

An **Gradient Descent** is an optimization algorithm used to minimize the loss function by iteratively updating the model's weights and biases in the direction that reduces prediction error..

Why Do We Need?

- Minimizes the loss function
- Improves model accuracy
- Helps neural networks learn from data
- Finds the optimal weights and biases

How It Works?

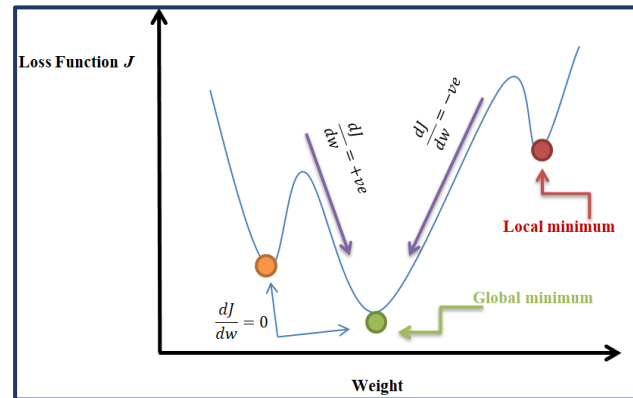
- Make a prediction using Forward Propagation.
- Calculate the error using a Loss Function.
- Compute gradients using Backpropagation.
- Update weights to reduce the error.
- Repeat until the loss is minimized.

Weight Update Rule

$$w_{new} = w_{old} - \eta \frac{\partial J}{\partial w}$$

Where:

- **w** = Weight
- **η (eta)** = Learning Rate
- **J** = Loss/Cost Function
- **$\partial J / \partial w$** = Gradient (slope)



- 📌 *Gradient = Direction of steepest increase in loss*
- 📌 *Gradient Descent moves in the opposite direction to minimize loss*
- 📌 *The goal is to find the weights that produce the lowest possible error*

Regularization — Preventing Overfitting

What is Overfitting?

A model learns the training data too well, including noise and irrelevant patterns, resulting in high training accuracy but poor performance on unseen data.

Common Regularization Techniques:

◆ Dropout

- Randomly deactivates neurons during training.
- Prevents dependency on specific neurons.
- Improves model generalization.

◆ Batch Normalization

- Normalizes activations within each layer.
- Stabilizes and accelerates training.
- Can also help reduce overfitting.

◆ Early Stopping

- Monitors validation performance during training.
- Stops training when validation loss stops improving.
- Prevents the model from memorizing training data.

◆ Weight Decay (L2 Regularization)

- Adds a penalty for large weights.
- Encourages simpler and more robust models.
- Reduces model complexity and overfitting.

 **Overfitting** = Memorization of training data

 **Regularization** = Better generalization to unseen data

Demo - I

From Mathematics to Implementation

Types of Neural Networks

- ❖ Artificial Neural Networks (ANN)
 - ✓ Foundation of Deep Learning
 - ✓ Best for structured/tabular data
- ❖ Convolutional Neural Networks (CNN)
 - ✓ Designed for image and computer vision tasks
- ❖ Recurrent Neural Networks (RNN)
 - ✓ Designed for sequential and time-series data
- ❖ Long Short-Term Memory (LSTM)
 - ✓ Advanced RNN architecture for long-term dependencies

Artificial Neural Network (ANN)

What is ANN?

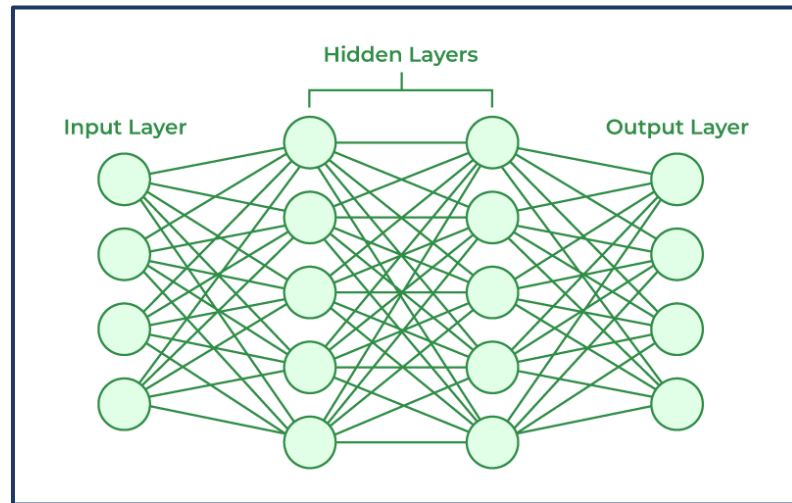
An Artificial Neural Network (ANN) is the fundamental building block of deep learning, inspired by the structure of biological neurons.

Key Characteristics

- Consists of Input, Hidden, and Output Layers
- Learns relationships between input features and outputs
- Suitable for structured/tabular data

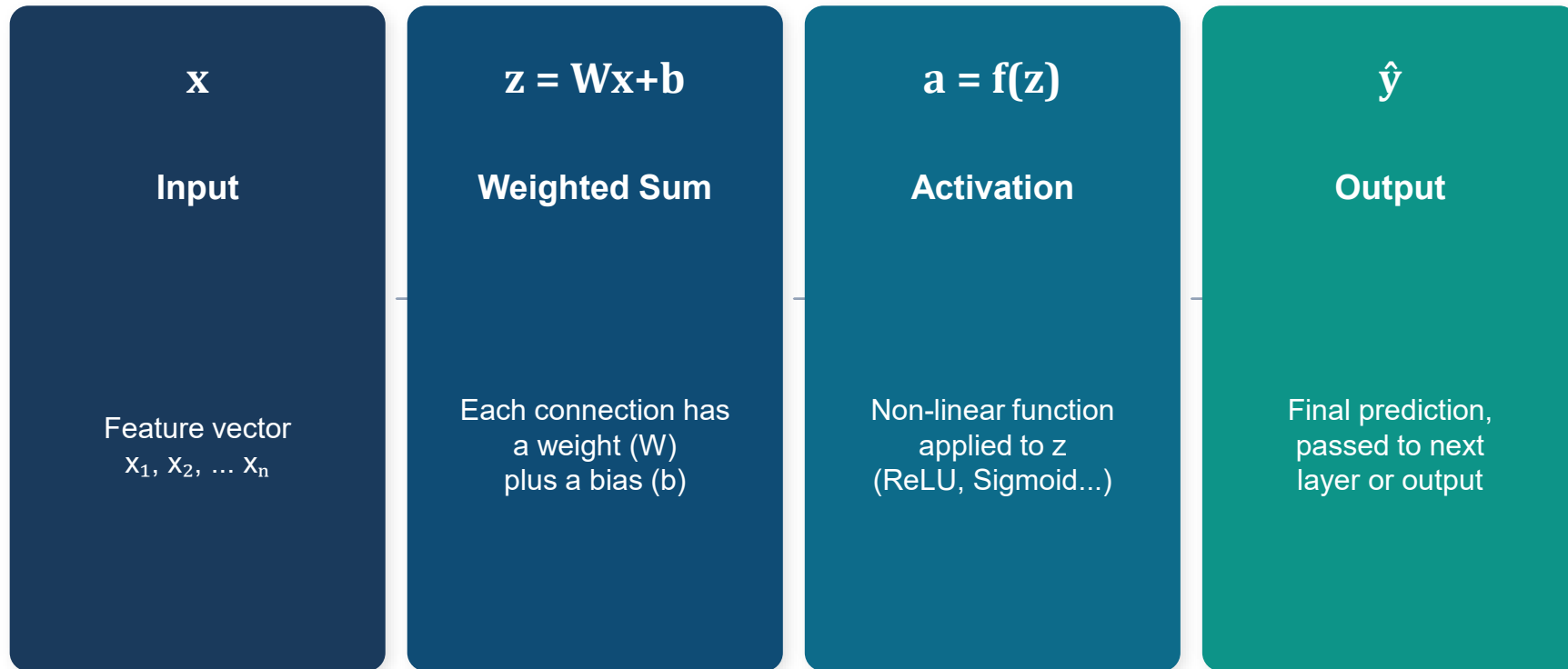
Common Applications

- Customer Churn Prediction
- Loan Approval
- Sales Forecasting
- House Price Prediction



✦ **ANN** is the foundation of all modern neural network architectures.

ANN — Layer-by-Layer, With Weights & Biases



ANN — How Training Actually Happens



Forward Pass

Input $x = 2.0$



Compute Loss

$$Loss = (y - \hat{y})^2$$



Backward Pass

$\partial Loss / \partial w$ calculated
via chain rule
for every weight



Update Weights

repeat next batch

Convolutional Neural Networks (CNNs)

What is CNN?

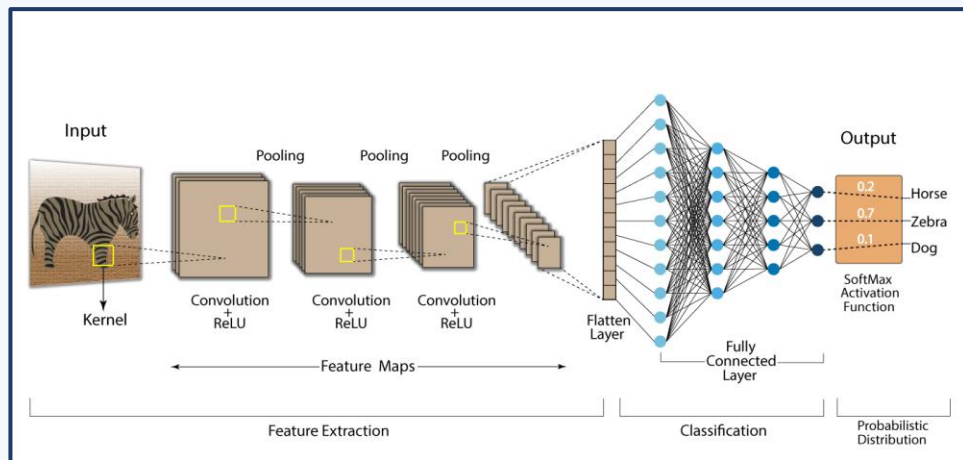
A Convolutional Neural Network (CNN) is a specialized neural network designed to process and analyze image data.

Key Characteristics

- Automatically extracts visual features
- Detects edges, shapes, textures, and objects
- Reduces the need for manual feature engineering

Common Applications

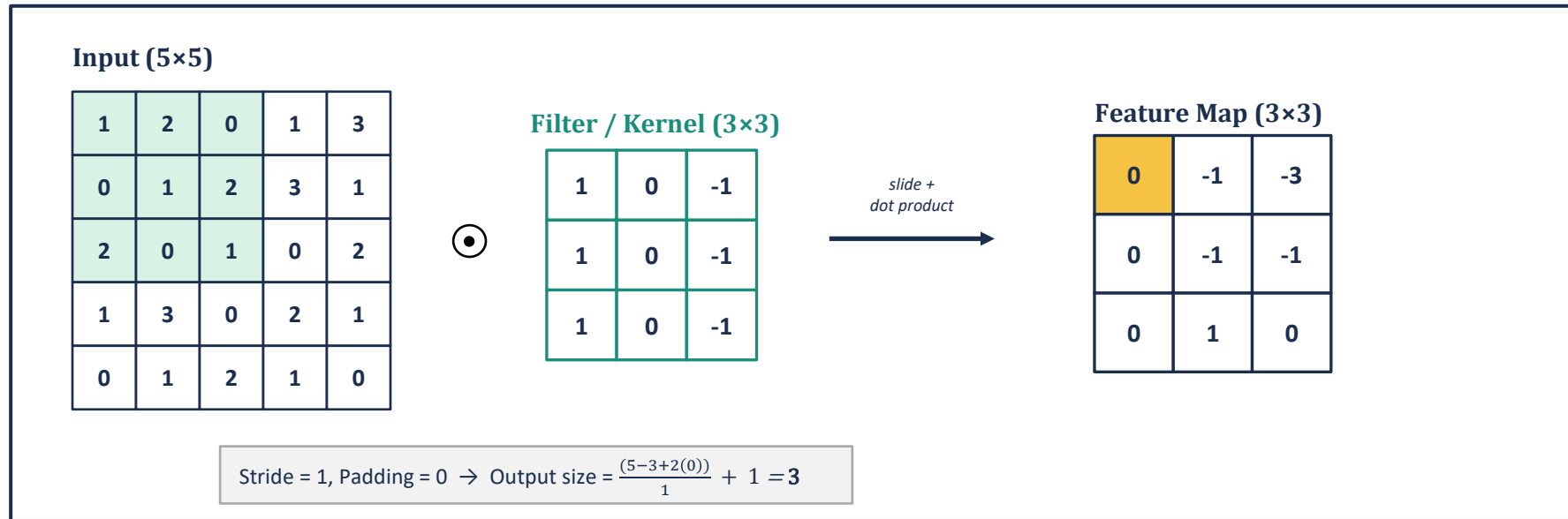
- Image Classification
- Face Recognition
- Medical Imaging



💡 CNNs exploit the spatial structure of images — nearby pixels are more related than distant ones.

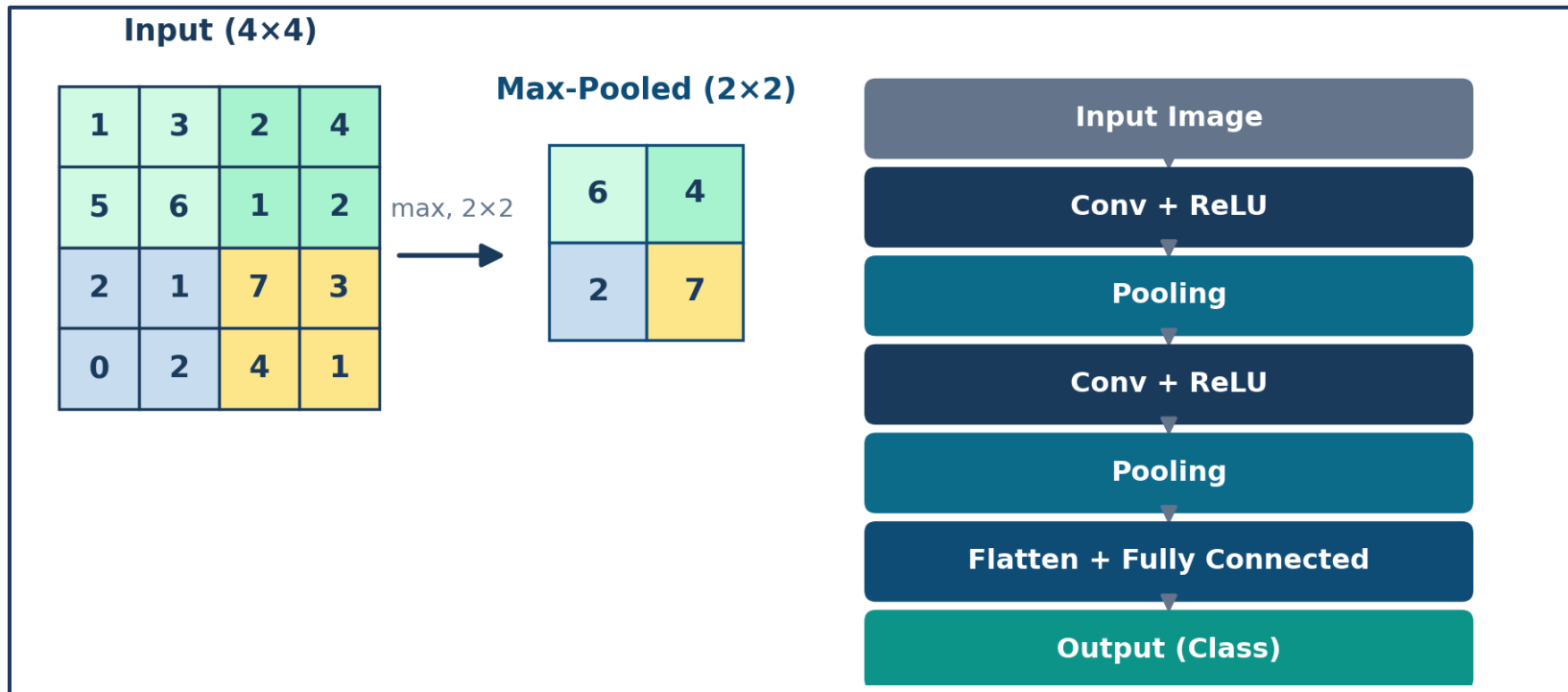
CNN — The Convolution Operation

A filter (kernel) slides across the input, computing a dot product at each position to produce a feature map — this is how a CNN detects edges, textures, and shapes.



Stride controls how far the filter moves each step; padding controls whether the output stays the same size as the input.

CNN — Pooling & the Full Architecture Stack



Pooling shrinks feature maps while keeping the strongest signals — stacking Conv+Pool blocks builds from edges to full objects.

Recurrent Neural Network (RNN)

What is RNN?

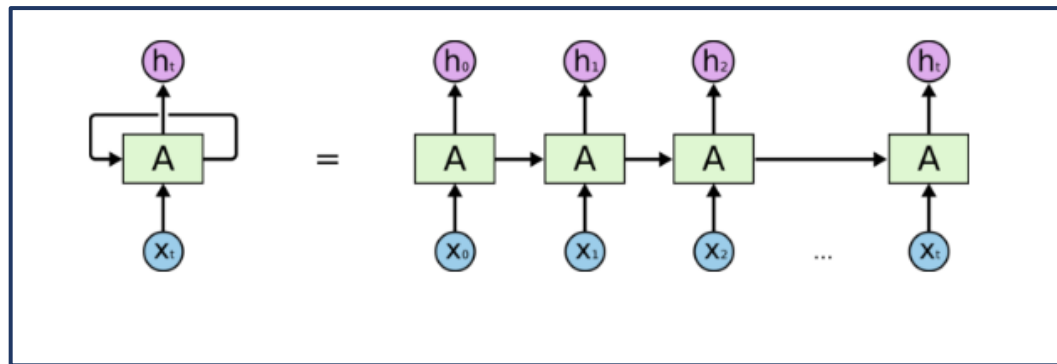
A Recurrent Neural Network (RNN) is designed to process sequential data by retaining information from previous inputs.

Key Characteristics

- Maintains memory of past inputs
- Suitable for sequence and time-series data
- Processes data step-by-step

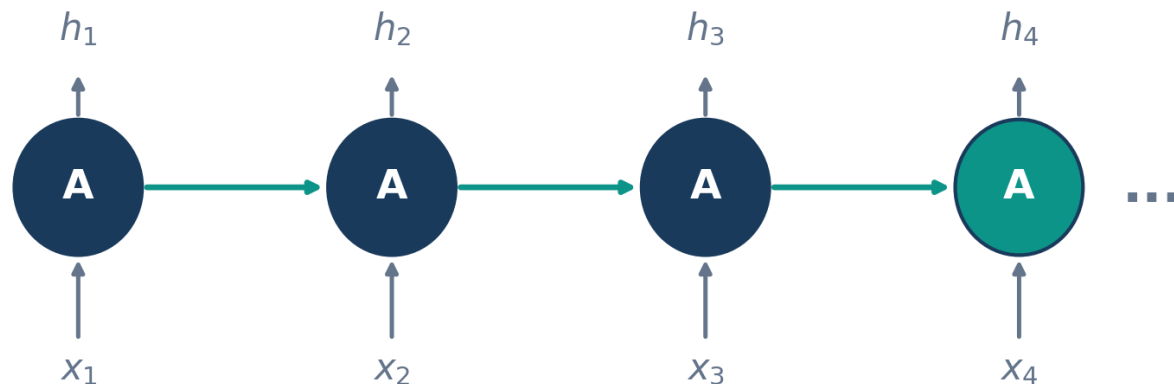
Common Applications

- Sentiment Analysis
- Language Modeling
- Time Series Forecasting
- Speech Processing



 *RNNs introduced the concept of memory in neural networks.*

RNN — Unrolled Through Time



Same cell 'A' and the same weights are reused at every timestep — the hidden state carries memory forward

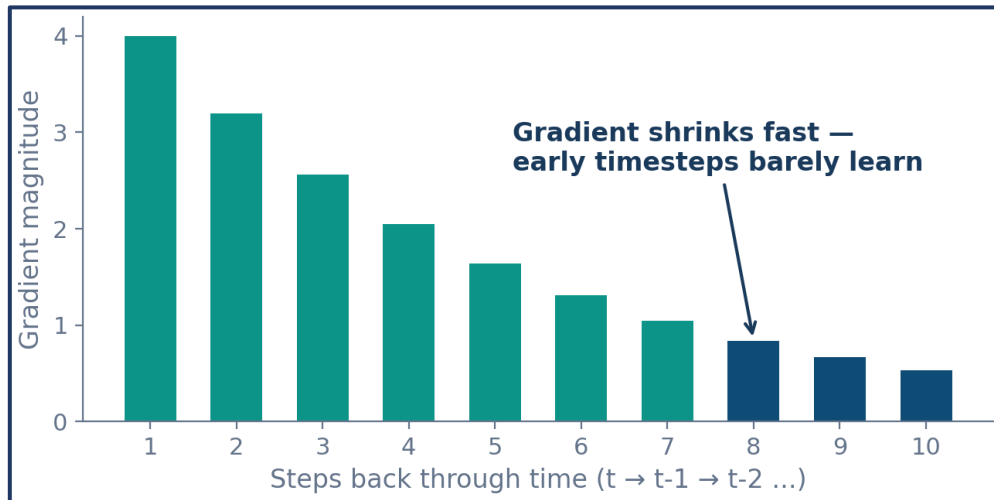
It's drawn as a chain, but it's the SAME cell and weights reused at every timestep — that sharing is what lets an RNN handle sequences of any length.

RNN — The Vanishing Gradient Problem

During backpropagation through time, gradients are multiplied at every step going backward.

When those multiplied values are less than 1, the gradient shrinks exponentially — earlier timesteps end up with almost no learning signal at all.

This is exactly the problem LSTMs were built to solve.



Coming up next: how LSTM's gates and cell state fix this.

Long Short-Term Memory (LSTM)

What is LSTM?

Long Short-Term Memory (LSTM) is an advanced version of RNN designed to remember information over long sequences.

Key Characteristics

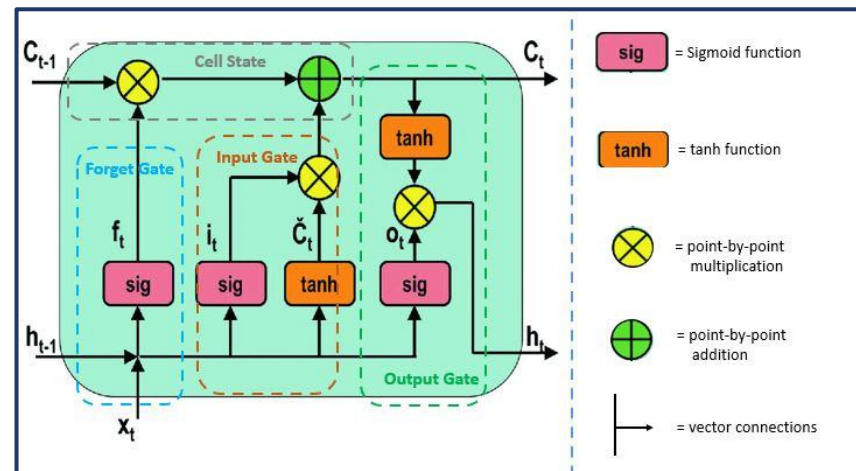
- Handles long-term dependencies
- Reduces the vanishing gradient problem
- Uses gates to control information flow

Common Applications

- Machine Translation
- Speech Recognition
- Text Generation

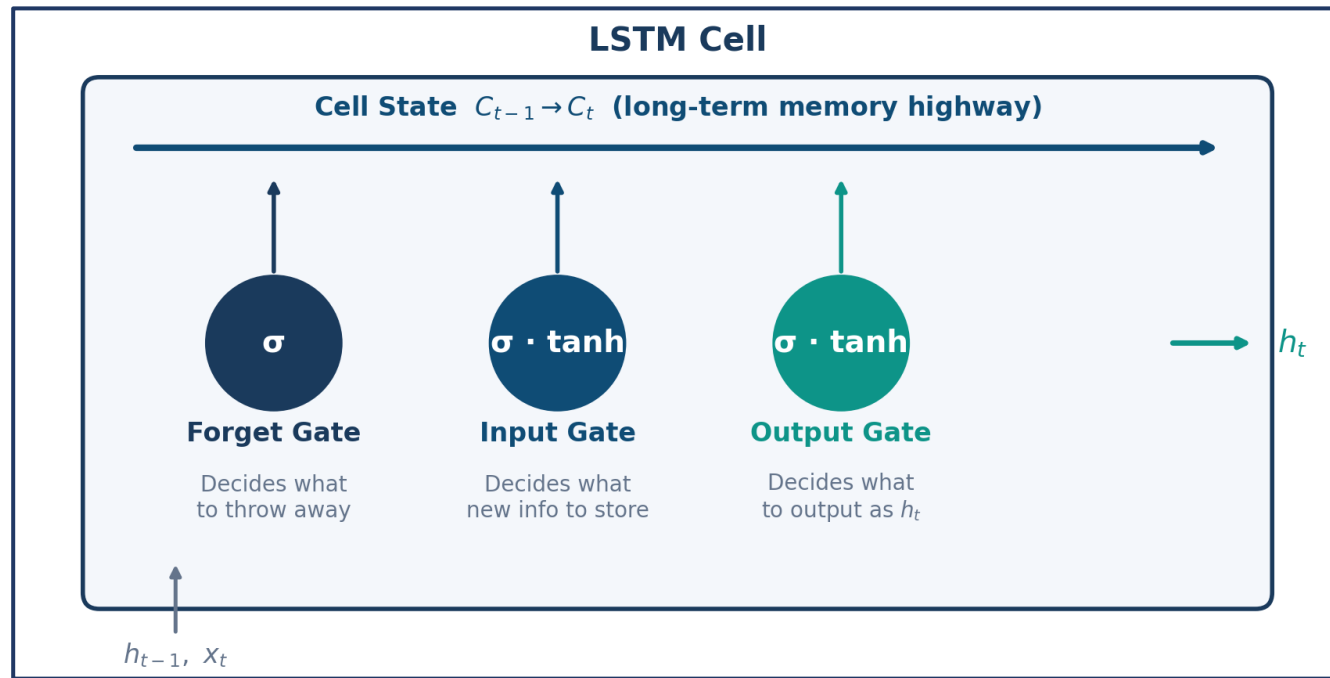
Components

- ✓ Forget Gate
- ✓ Input Gate
- ✓ Output Gate



📌 *LSTMs improve sequence learning by retaining important information for longer periods.*

LSTM — Inside the Cell



The cell state is a conveyor belt for long-term memory; the three gates decide what's forgotten, added, and exposed at each step.

LSTM vs RNN — What LSTM Fixes

Both process sequential data, but a plain RNN carries only a single hidden state forward. LSTM adds a separate cell state plus three gates to control it.

Feature	RNN	LSTM
Long-Term Dependencies	Struggles (vanishing gradient)	Handles well (cell state)
Memory Mechanism	Single hidden state	Cell state + hidden state
Gating	None	Forget / Input / Output gates
Training Stability	Prone to vanishing/exploding gradients	More stable gradients
Parameter Count	Fewer parameters	~4x more parameters per cell
Common Use Cases	Short sequences, simple time-series	Long text, speech, long time-series
Computation Cost	Lower	Higher
Typical Choice Today	Rarely used alone	Preferred over vanilla RNN

Summary

Neural Network	Best For	Example Applications
ANN	Structured Data	Churn Prediction, Regression
CNN	Images	Face Recognition, Medical Imaging
RNN	Sequential Data	Sentiment Analysis, Forecasting
LSTM	Long Sequences	Translation, Speech Recognition

Key Takeaways — What You Learned Today

❑ **Neurons & Networks**

- ✓ Artificial neurons are inspired by biological neurons.
- ✓ Neural networks learn through weights, biases, activation functions, and layers.

❑ **Training Loop**

- ✓ Forward Propagation generates predictions.
- ✓ Loss Functions measure errors.
- ✓ Backpropagation & Optimizers update weights to improve performance.

❑ **Prevent Overfitting**

- ✓ Techniques such as Dropout, Batch Normalization, and Early Stopping help models generalize better to unseen data.

❑ **Neural Network Architectures**

- ✓ ANNs for structured data.
- ✓ CNNs for images and computer vision.
- ✓ RNNs/LSTMs for sequential and time-series data.

❑ **Practical Power**

- ✓ Pre-trained models enable developers to solve real-world problems with minimal code.

Master the fundamentals, understand the architectures, and leverage pre-trained models to build intelligent AI solutions efficiently.

